PROGRAMMING IN JAVA ASSIGNMENT 2 – CO2

CO2 Coverage: Strings and StringBuilder, regular expressions, the Collection Framework – List, Queue, Deque, PriorityQueue, Set and Map, the hashCode–equals contract, Generics, Comparable and Comparator, and the Collections utility class (Sessions 15–26).

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/Pavithra16780/CSA09-192472155-Assignment2.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time

complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? 6 Marks

Bloom's Level: K4 – Analyse

Problem Context – Railway e-Ticket Generation

A railway reservation system generates an e-ticket for every booking. Each e-ticket is built by joining many fields – PNR, passenger names, coach, berth, fare and journey details – into a single printable text block. The current code builds the ticket using the + operator inside a loop.

1. Aim

To compare String, StringBuilder and StringBuffer while generating railway e-tickets and identify the most suitable approach based on execution time, memory usage and thread safety.

2. Problem Explanation

A railway reservation system generates tickets containing PNR number, passenger name, train number, coach and berth number, fare, and journey date and time. Repeatedly joining these fields using the + operator may create many intermediate String objects.

3. Prediction Before Implementation

Concept	Prediction	Reason
String	Likely slower for repeated concatenation.	String is immutable; concatenation creates new String values.
StringBuilder	Expected to be fastest for single-threaded building.	Mutable buffer; append operations avoid repeatedly creating complete strings.

StringBuffer	Usually slightly slower than StringBuilder.	Mutable and synchronized methods provide thread safety for individual operations.

4. Experimental Workloads

Workload	Number of tickets	Fields per ticket
1	1	6
2	10,000	50
3	50,000	100

Measure execution time with System.nanoTime(). Run each approach several times, warm up the JVM where possible, and calculate an average. Estimate intermediate object creation from the algorithm; exact object counts and memory use require a profiler or JVM measurement tool.

5. Java Program

```
public class Q1Benchmark {

static String usingString(int fields) {

String ticket = "";

for (int i = 1; i <= fields; i++) {

ticket = ticket + " Field" + i;

}

return ticket;

}

static String usingStringBuilder(int fields) {

StringBuilder ticket = new StringBuilder();
```

```
for (int i = 1; i <= fields; i++) {

ticket.append(" Field").append(i);

}

return ticket.toString();

}

static String usingStringBuffer(int fields) {

StringBuffer ticket = new StringBuffer();

for (int i = 1; i <= fields; i++) {

ticket.append(" Field").append(i);

}

return ticket.toString();

}

static void test(String name, int tickets, int fields) {

long start = System.nanoTime();

for (int i = 0; i < tickets; i++) {

if (name.equals("String")) {

usingString(fields);

} else if (name.equals("StringBuilder")) {

usingStringBuilder(fields);

} else {

usingStringBuffer(fields);

}
```

```
}

long end = System.nanoTime();

System.out.println(name + " : " +

(end - start) / 1_000_000.0 + " ms");

}

public static void main(String[] args) {

int[][] workloads = {

{1, 6}, {10000, 50}, {50000, 100}

};

for (int i = 0; i < workloads.length; i++) {

int tickets = workloads[i][0];

int fields = workloads[i][1];

System.out.println("\nWorkload " + (i + 1));

test("String", tickets, fields);

test("StringBuilder", tickets, fields);

test("StringBuffer", tickets, fields);

}

}

}
```

6. Sample Output Format

```
Workload 1

String : [measured time] ms
```

StringBuilder : [measured time] ms

StringBuffer : [measured time] ms

Workload 2

String : [measured time] ms

StringBuilder : [measured time] ms

StringBuffer : [measured time] ms

Workload 3

String : [measured time] ms

StringBuilder : [measured time] ms

StringBuffer : [measured time] ms

The values above are placeholders, not benchmark results. Replace them with the measurements from your own execution.

7. Comparison

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Synchronization	Not applicable to immutable value	Not synchronized	Synchronized methods
Repeated append performance	Often slower	Usually fastest	Usually slower than StringBuilder
Best use	Fixed text	Local text construction	Shared mutable buffer where synchronized methods are needed

8. Design Decision

Per-ticket generation: use StringBuilder because each ticket can be built independently.

Shared booking log: StringBuffer can be used when multiple threads append to the same buffer and its synchronized methods meet the requirement.

For production systems, a dedicated thread-safe logging framework is generally preferable to sharing one StringBuffer.

9. Alternative Concept and Complexity Analysis

String.join(), StringJoiner and String.format() are alternatives for composing text. StringJoiner is useful when fields need a delimiter. String.join() is convenient for joining a collection of values. String.format() is readable for formatted output but may add formatting overhead.

```
import java.util.StringJoiner;

public class AlternativeTicket {

public static void main(String[] args) {

StringJoiner ticket = new StringJoiner(" | ");

ticket.add("PNR: 12345");

ticket.add("Name: Arun");

ticket.add("Coach: A1");

ticket.add("Berth: 25");

ticket.add("Fare: 500");

System.out.println(ticket);

}

}
```

Approach	Typical complexity	Space consideration
Repeated concatenation	$O(n^2)$ character copying in the general growing-string case	Many intermediate strings may be allocated.
StringBuilder	O(n) amortized for appending n total characters	Resizable buffer plus final String.
StringJoiner / String.join	O(n) in total output size for ordinary joining	Stores/constructs joined content.
String.format	Depends on format and conversion work; often higher constant overhead	Formatting machinery and resulting String.

Here n refers to the total number of output characters. Repeated concatenation has quadratic copying when each step copies an increasingly long accumulated result.

10. Result

String, StringBuilder and StringBuffer can be compared using the designed workloads. StringBuilder is recommended for individual ticket construction because it supports efficient mutable text operations. StringBuffer provides synchronized methods for shared mutable text, although a logging framework is preferable in many production systems

---

Q2 – When Is ArrayList an Appropriate Engineering Choice? 4 Marks

Bloom's Level: K4 – Analyse

Problem Context – Music Playlist Editor

A music-streaming app lets users build playlists. Users add songs at the end, insert songs at chosen positions, remove songs and jump directly to the k-th song. The app currently stores every playlist in an ArrayList<Song>.

1. Aim

To compare ArrayList and LinkedList for playlist operations and determine when ArrayDeque is a better choice.

2. Problem Explanation

A music-streaming application allows users to add songs at the end, insert songs at selected positions, remove songs, access the kth song and display the entire playlist.

3. Java Program

```
import java.util.*;

public class Q2Comparison {

public static void main(String[] args) {

ArrayList<String> arrayList = new ArrayList<>();

LinkedList<String> linkedList = new LinkedList<>();

String[] songs = {

"Song A", "Song B", "Song C", "Song D", "Song E"

};

for (String song : songs) {

arrayList.add(song);

linkedList.add(song);

}

System.out.println("ArrayList: " + arrayList);

System.out.println("LinkedList: " + linkedList);

arrayList.add("Song F");

linkedList.add("Song F");

System.out.println("\nAfter adding at end:");

System.out.println("ArrayList: " + arrayList);
```

```
System.out.println("LinkedList: " + linkedList);

arrayList.add(2, "New Song");

linkedList.add(2, "New Song");

System.out.println("\nAfter inserting in middle:");

System.out.println("ArrayList: " + arrayList);

System.out.println("LinkedList: " + linkedList);

arrayList.remove(0);

linkedList.remove(0);

System.out.println("\nAfter removing first song:");

System.out.println("ArrayList: " + arrayList);

System.out.println("LinkedList: " + linkedList);

System.out.println("\nThird song:");

System.out.println(arrayList.get(2));

System.out.println(linkedList.get(2));

ArrayDeque<String> queue = new ArrayDeque<>();

queue.addLast("Song A");

queue.addLast("Song B");

queue.addFirst("Song C");

System.out.println("\nUp Next Queue: " + queue);

queue.removeFirst();

System.out.println("After playing first song: " + queue);

}

}
```

4. Sample Output

```
ArrayList: [Song A, Song B, Song C, Song D, Song E]

LinkedList: [Song A, Song B, Song C, Song D, Song E]

After adding at end:

ArrayList: [Song A, Song B, Song C, Song D, Song E, Song F]

LinkedList: [Song A, Song B, Song C, Song D, Song E, Song F]

After inserting in middle:

ArrayList: [Song A, Song B, New Song, Song C, Song D, Song E, Song F]

LinkedList: [Song A, Song B, New Song, Song C, Song D, Song E, Song F]

After removing first song:

ArrayList: [Song B, New Song, Song C, Song D, Song E, Song F]

LinkedList: [Song B, New Song, Song C, Song D, Song E, Song F]

Third song:

Song C

Song C

Up Next Queue: [Song C, Song A, Song B]

After playing first song: [Song A, Song B]
```

5. Time Complexity Comparison

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized
Insert at front	O(n)	O(1)	O(1) amortized
Insert in middle	O(n)	O(n) to locate; O(1) to link once located	O(n)
Get kth element	O(1)	O(n)	O(n) traversal
Iterate	O(n)	O(n)	O(n)

6. Experimental Design

Test with 1,000, 10,000 and 100,000 songs. Measure adding at the end, inserting/removing at the front and middle, accessing an indexed element where supported, and iterating through the entire list. Use System.nanoTime(), repeat the measurements and calculate averages.

7. Engineering Decision

ArrayList is appropriate when indexed access and iteration are frequent, additions are usually at the end, and front/middle modifications are uncommon.

LinkedList can be suitable for frequent end operations, but indexed access is slow and locating a middle position still takes O(n).

ArrayDeque is appropriate for an Up Next queue with additions at both ends and removals from the front.

8. Alternative Concept – Plain Array

A plain array has fixed capacity. Accessing the kth element takes O(1), while insertion or deletion generally requires shifting elements and takes O(n). Iteration takes O(n). Resizing requires allocating and copying to a larger array.

9. Result

ArrayList is a reasonable engineering choice for normal playlists that need frequent indexed access. ArrayDeque is more suitable for the changed Up Next requirement involving frequent operations at both ends.

Q3 – Engineer the hashCode–equals Contract **6 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Event Attendance De-duplication

A college fest records attendance by scanning student ID cards at several gates. The same student may be scanned more than once. The organisers store scans in a HashSet<Student> to keep only unique students. A junior developer has written the Student class below.

```
class Student {
    String regNo; String name;
    Student(String regNo, String name) { this.regNo = regNo; this.name =
    name; } public boolean equals(Object o) { if (!(o instanceof Student))
    return false; return regNo.equals(((Student) o).regNo);
    }
    // hashCode() is not overridden
}
```

1. Aim

To understand how equals() and hashCode() work together in HashSet and how they prevent duplicate student attendance records.

2. Problem Explanation

In a college fest, students scan their ID cards at different entry gates. The same student may scan more than once. A HashSet is used to store unique students based on registration number.

3. Important Concepts

equals() checks whether two objects are logically equal.

hashCode() returns an integer hash value for an object.

If two objects are equal according to equals(), they must return the same hash code.

Different objects may have the same hash code; HashSet also uses equals() to check equality.

4. Complete the Code

```
import java.util.HashSet;

import java.util.Set;

class Student {
```

```
private final String regNo;

private final String name;

Student(String regNo, String name) {

this.regNo = regNo;

this.name = name;

}

@Override

public boolean equals(Object o) {

if (this == o) return true;

if (!(o instanceof Student)) return false;

return regNo.equals(((Student) o).regNo);

}

// TODO 1 completed

@Override

public int hashCode() {

return regNo.hashCode();

}

@Override

public String toString() {

return regNo + " " + name;

}
```

```
}

public class Attendance {

public static void main(String[] args) {

Set<Student> present = new HashSet<>();

present.add(new Student("192311001", "Arun"));

present.add(new Student("192311002", "Divya"));

present.add(new Student("192311001", "Arun"));

System.out.println("Unique students: " + present.size());

}

}
```

5. Expected Output

```
Unique students: 2
```

6. Output Before and After Completing TODO 1

With equals() overridden but hashCode() not overridden, two logically equal Student objects may be stored as separate entries because they can be placed in different hash buckets. The exact output depends on the hash codes assigned by the JVM. After overriding hashCode() using regNo, equal registration numbers produce the same hash code and HashSet can identify the duplicate.

7. Three Experimental Tests

Test	Input	Expected size after correction
1	Arun and Divya with different registration numbers	2

2	Arun, Divya and Arun scanned again	2
3	Three different registration numbers	3

8. What Happens When a Field Changes?

If a field used in hashCode() changes after insertion into a HashSet, the object may remain in the bucket selected using its old hash code. Searching for or removing it may fail. This is why regNo is declared final in this program.

9. Alternative Concept – TreeSet and HashMap

A TreeSet can use a Comparator that compares registration numbers. A HashMap can use registration number as the key, so each key identifies one student.

```
// TreeSet: assumes Student has a getRegNo() method

TreeSet<Student> students =

new TreeSet<>(Comparator.comparing(Student::getRegNo));

// HashMap alternative

HashMap<String, Student> map = new HashMap<>();

map.put("192311001", new Student("192311001", "Arun"));

map.put("192311002", new Student("192311002", "Divya"));

System.out.println(map.size());
```

10. Complexity Analysis

Collection	Average insertion/search	Space
HashSet	O(1) average	O(n)
TreeSet	O(log n)	O(n)

HashMap	O(1) average	O(n)

The TreeSet code above requires the Student class to expose a getRegNo() method. The supplied Student class can be extended with public String getRegNo() { return regNo; }.

11. Result

Overriding both equals() and hashCode() using registration number provides correct student de-duplication in HashSet.

Q4 – Choosing a Map for a Pharmacy Inventory **6 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Hospital Pharmacy Stock Lookup

A hospital pharmacy keeps stock for each medicine, identified by a medicine code. Pharmacists look up stock by code hundreds of times a day, and every evening the system prints a report of all medicines in code order and a list of medicines in the order they were added.

1. Aim

To compare HashMap, TreeMap and LinkedHashMap for storing medicine inventory and selecting the appropriate Map for stock lookup and reporting.

2. Problem Explanation

A hospital pharmacy stores medicine information using a unique medicine code. It must search stock by code and generate reports in both medicine-code order and insertion order.

3. Java Program

```
import java.util.*;

public class Q4Inventory {

public static void main(String[] args) {

HashMap<String, Integer> hashMap = new HashMap<>();

TreeMap<String, Integer> treeMap = new TreeMap<>();

LinkedHashMap<String, Integer> linkedMap =
```

```
new LinkedHashMap<>();
String[] codes = {
"M103", "M101", "M105", "M102", "M104"
};
int[] stock = {50, 100, 30, 75, 60};
for (int i = 0; i < codes.length; i++) {
hashMap.put(codes[i], stock[i]);
treeMap.put(codes[i], stock[i]);
linkedMap.put(codes[i], stock[i]);
}
System.out.println("HashMap Report:");
System.out.println(hashMap);
System.out.println("\nTreeMap Report:");
System.out.println(treeMap);
System.out.println("\nLinkedHashMap Report:");
System.out.println(linkedMap);
System.out.println("\nMedicine Lookup:");
String searchCode = "M103";
if (hashMap.containsKey(searchCode)) {
System.out.println(searchCode + " Stock: "
+ hashMap.get(searchCode));
} else {
System.out.println("Medicine not found");
}
```

```
}

}
```

4. Sample Output

```
HashMap Report:

{M101=100, M102=75, M103=50, M104=60, M105=30}

TreeMap Report:

{M101=100, M102=75, M103=50, M104=60, M105=30}

LinkedHashMap Report:

{M103=50, M101=100, M105=30, M102=75, M104=60}

Medicine Lookup:

M103 Stock: 50
```

Note: HashMap does not guarantee any iteration order. Its displayed order above is illustrative. TreeMap sorts keys; LinkedHashMap preserves insertion order.

5. Comparison Table

Feature	HashMap	TreeMap	LinkedHashMap
Ordering	No guaranteed order	Sorted by key	Insertion order by default
put()	O(1) average	O(log n)	O(1) average
get()	O(1) average	O(log n)	O(1) average
Iteration	O(n)	O(n)	O(n)
Null key	One null key allowed	Normally not allowed with natural ordering	One null key allowed
Main use	Fast lookup	Sorted reports	Insertion-order reports

6. Experimental Design

Test with 1,000, 10,000 and 100,000 medicines. Measure insertion time, time for 100,000 lookups, code-order report generation and insertion-order report generation. Use the same keys and lookup sequence for each Map, repeat the experiment and calculate averages.

7. Scalability Analysis and Design Decision

HashMap provides fast average stock lookup.

TreeMap automatically maintains medicine codes in sorted order.

LinkedHashMap maintains insertion order.

Use HashMap for lookup-heavy work, TreeMap when sorted reports are central, and LinkedHashMap when insertion-order reports are required. A system may maintain more than one view if both report orders are frequent.

8. Why Hashtable Is Not Preferred

Hashtable is a legacy collection.

Its methods are synchronized.

It does not allow null keys or values.

Modern alternatives such as HashMap and ConcurrentHashMap are generally preferred according to the thread-safety requirement.

9. Alternative Concept – Sorted Array

Operation	Sorted array approach
Binary search lookup	O(log n)
Insertion	O(n), due to shifting
Deletion	O(n), due to shifting
Sorted report	O(n)
Storage	O(n)

A sorted array supports efficient binary-search lookup, but insertion and deletion require shifting elements. Arrays.binarySearch() can be used to locate a medicine code.

10. Result

HashMap is suitable for fast inventory lookup, TreeMap for sorted reports, and LinkedHashMap for preserving insertion order. The final selection depends on the pharmacy's main operational requirement.

Q5 – Derive a Generic Ranking Component from a Principle **7 Marks**

Bloom's Level: K4 – Analyse

Problem Context – Inter-College Sports Leaderboard

A sports meet ranks teams by points (higher first). Ties are broken by goal difference (higher first), then by team name (alphabetical). The same ranking component will later be reused for ranking players, colleges and quiz teams.

1. Aim

To develop a reusable Java leaderboard using Generics and Comparator to rank sports teams based on points, goal difference and team name.

2. Problem Explanation

An inter-college sports competition ranks teams using three rules: higher points first; if points are equal, higher goal difference first; if both are equal, team name in alphabetical order.

3. Important Concepts

Generics allow a class to work with different data types.

Comparable defines a type's natural ordering.

Comparator defines custom ordering separately from the class.

Wildcards allow flexible generic type references.

thenComparing() adds further sorting conditions.

4. Complete the Code

```
import java.util.ArrayList;
```

```
import java.util.Comparator;

import java.util.List;

class Team {

final String name;

final int points;

final int goalDiff;

Team(String name, int points, int goalDiff) {

this.name = name;

this.points = points;

this.goalDiff = goalDiff;

}

@Override

public String toString() {

return name + "(" + points + "," + goalDiff + ")";

}

}

class Leaderboard<T> {

private final List<T> items = new ArrayList<>();

private final Comparator<? super T> order;

Leaderboard(Comparator<? super T> order) {

this.order = order;
```

```
}
void add(T item) {
items.add(item);
}
List<T> ranking() {
// TODO 1 completed
List<T> result = new ArrayList<>(items);
result.sort(order);
return result;
}
}
public class LeaderboardDemo {
public static void main(String[] args) {
// TODO 2 completed
Comparator<Team> byRank =
Comparator.comparingInt((Team t) -> t.points)
.reversed()
.thenComparing(
Comparator.comparingInt(
(Team t) -> t.goalDiff
).reversed()
```

```
)
.thenComparing(t -> t.name);
Leaderboard<Team> board =
new Leaderboard<>(byRank);
board.add(new Team("Panthers", 9, 4));
board.add(new Team("Eagles", 9, 6));
board.add(new Team("Cobras", 6, 1));
board.add(new Team("Bisons", 9, 4));
System.out.println(board.ranking());
}
}
```

5. Expected Output

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

6. Explanation of the Output

Eagles have 9 points and goal difference 6, so they rank first.

Bisons and Panthers both have 9 points and goal difference 4.

Their points and goal differences are equal, so alphabetical order is applied.

Bisons comes before Panthers.

Cobras has 6 points, so it comes last.

7. Second Test Case

Add three more teams to verify ties at each comparison level:

```
board.add(new Team("Lions", 12, 5));

board.add(new Team("Tigers", 12, 8));

board.add(new Team("Wolves", 12, 8));
```

The three added teams should be ordered as Tigers(12,8), Wolves(12,8), Lions(12,5), because goal difference is compared before name.

8. Why the Comparator Is Correct

The comparator follows a fixed sequence: points descending, goal difference descending, and name ascending. These comparisons use consistent integer and String orderings. The combined lexicographic order is transitive. Teams with identical values for all three fields compare as equal.

9. Alternative Concept – Comparable

If Team implemented Comparable<Team>, its compareTo() method could define the default ranking. Comparator is more flexible because the same Team objects can be ranked differently without modifying the Team class. It also supports reuse for players and colleges.

10. Complexity Analysis

Method	Time complexity	Extra space
Comparator-based sorting	O(n log n)	O(n) copied list plus sorting workspace
Comparable-based sorting	O(n log n)	Sorting workspace
Three stable sorts	Three O(n log n) sorts, still O(n log n) asymptotically	Depends on sorting implementation

The ranking() method copies the original items and sorts the copy, so the stored insertion list is not modified.

11. Result

The generic Leaderboard class successfully ranks teams using a Comparator. The design can be reused for players, colleges and quiz teams by supplying suitable comparators.

Q6 – Priority, Order and Fairness in a Queue **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Hospital Emergency Triage

Patients arriving at an emergency department are given a severity level from 1 (critical) to 5 (minor). The doctor on duty must always see the most severe patient next; among patients with the same severity, the one who arrived first must be seen first.

1. Aim

To implement a hospital emergency triage system using PriorityQueue and Comparator, ensuring that critical patients are treated first and patients with equal severity are treated in arrival order.

2. Problem Explanation

Patients receive severity levels from 1 (critical) to 5 (minor). The patient with the lowest severity number must be treated first. If two patients have equal severity, the earlier arrival must be treated first.

3. Java Program – Complete the Code

```
import java.util.Comparator;

import java.util.PriorityQueue;

class Patient {

final String name;

final int severity;

final long arrival;

Patient(String name, int severity, long arrival) {
```

```
this.name = name;
this.severity = severity;
this.arrival = arrival;
}
@Override
public String toString() {
return name + "(S" + severity + ")";
}
}
public class Triage {
private long counter = 0;
// TODO 1 completed
PriorityQueue<Patient> queue =
new PriorityQueue<>(
Comparator.comparingInt((Patient p) -> p.severity)
.thenComparingLong(p -> p.arrival)
);
void admit(String name, int severity) {
queue.add(new Patient(name, severity, counter++));
}
Patient next() {
return queue.poll();
}
public static void main(String[] args) {
```

```
Triage er = new Triage();

er.admit("Ravi", 3);

er.admit("Meena", 1);

er.admit("John", 3);

er.admit("Asha", 5);

er.admit("Kumar", 1);

er.admit("Lata", 2);

StringBuilder order = new StringBuilder();

Patient p;

while ((p = er.next()) != null) {

order.append(p).append(' ');

}

System.out.println(order.toString().trim());

}

}
```

4. Expected Output

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

5. Explanation

Patient	Severity	Arrival order
Ravi	3	1
Meena	1	2
John	3	3
Asha	5	4
Kumar	1	5

Lata	2	6

Meena and Kumar have equal severity. Meena arrived first, so she is treated first. Ravi and John also have equal severity, so Ravi is treated first.

6. Why Plain Queue Is Not Sufficient

A normal FIFO queue serves patients in arrival order and does not consider emergency severity. PriorityQueue serves patients according to the comparator and is therefore more appropriate for this rule.

7. Why Printing PriorityQueue Directly Is Not Enough

Printing a PriorityQueue directly does not guarantee that all elements appear in treatment order. To display the correct order, repeatedly call poll() to remove the next patient according to priority.

8. Growth Investigation

Test with 100, 1,000 and 10,000 patients. Measure insertion and removal time using System.nanoTime(). Repeat each measurement and calculate the average.

9. Complexity Analysis

Operation	PriorityQueue complexity
Admit patient	O(log n)
Remove next patient	O(log n)
View next patient	O(1)
Display by polling all patients	O(n log n)
Space	O(n)

10. Starvation Problem and Aging

A severity-5 patient may wait indefinitely if new critical patients continuously arrive. This is starvation. An aging policy can increase the priority of patients as their waiting time grows. A real hospital must apply aging without overriding clinically necessary emergency decisions.

11. Alternative Concept – Five FIFO Queues

The system can maintain five FIFO queues, one for each severity level. Admission to the relevant queue is O(1). To serve the next patient, check queues from severity 1 through severity 5. Since there are a fixed five queues, this check is O(1). A sorted LinkedList is another alternative, but finding the insertion position takes O(n); removing the first element takes O(1).

12. Deployment Decision

A PriorityQueue can efficiently manage a workload of 5,000 patients per day because insertion and removal take O(log n) time. A real hospital system also needs patient registration, persistent records, monitoring, emergency overrides and fairness policies.

13. Result

The PriorityQueue treats patients according to emergency severity and preserves arrival order among patients with equal severity.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate **13 Marks**

Bloom's Level: K5 – Evaluate

Review the evidence you generated while working on Q1–Q6. Select **ONE** solution that shows a significant limitation as the data size, usage pattern or requirement changes.

You are **not** permitted simply to replace one collection with another and state that it is faster. You must understand the problem first and then develop an appropriate redesign.

1. Aim

To identify performance limitations in the original railway ticket-generation system and redesign it using StringBuilder to reduce unnecessary String object creation and improve execution efficiency.

A. Diagnose the Bottleneck (2 Marks)

The original system uses the String + operator repeatedly inside a loop to generate ticket details.

String is immutable.

Every concatenation creates a new String value.

Previous string contents may be copied repeatedly.

Large numbers of tickets increase execution time.

Unnecessary intermediate objects increase memory pressure.

Main bottleneck: repeated String object creation and character copying during ticket generation.

B. Define the Design Objective (2 Marks)

Reduce unnecessary intermediate String allocations.

Improve ticket-generation performance.

Reduce repeated character copying.

Support large numbers of tickets.

Produce the same ticket information as the original system.

Measurable objective: compare execution time for generating 10,000 tickets containing 50 fields each using both approaches.

C. Design the Revised Solution (4 Marks)

Original solution: repeated String concatenation. Redesigned solution: create a StringBuilder, append each ticket field, and convert the completed buffer to a String.

Original system	Redesigned system
Initialize String	Create StringBuilder
Repeated concatenation using +	Append ticket fields
Many intermediate String values	Resizable mutable buffer
Produce final ticket	Convert buffer to final String

D. Predict Before Implementing (1 Mark)

StringBuilder should perform fewer intermediate allocations.

Character copying should be reduced.

Execution time should generally be lower for large workloads.

Temporary-string memory pressure should be reduced.

E. Implement and Validate the Redesign (2 Marks)

```
public class Q7Redesign {
// Original solution
static String original(int fields) {
String ticket = "";
for (int i = 1; i <= fields; i++) {
ticket = ticket + " Field" + i;
}
return ticket;
}
// Redesigned solution
static String redesigned(int fields) {
StringBuilder ticket = new StringBuilder();
for (int i = 1; i <= fields; i++) {
ticket.append(" Field").append(i);
}
return ticket.toString();
}
public static void main(String[] args) {
int tickets = 10000;
int fields = 50;
long start1 = System.nanoTime();
```

```
for (int i = 0; i < tickets; i++) {
original(fields);
}
long end1 = System.nanoTime();
long start2 = System.nanoTime();
for (int i = 0; i < tickets; i++) {
redesigned(fields);
}
long end2 = System.nanoTime();
System.out.println("Original Time: " +
(end1 - start1) / 1_000_000.0 + " ms");
System.out.println("Redesigned Time: " +
(end2 - start2) / 1_000_000.0 + " ms");
}
}
```

Experimental Result Table

Measurement	Original String (ms)	Redesigned StringBuilder (ms)
Run 1	Enter actual result	Enter actual result
Run 2	Enter actual result	Enter actual result
Run 3	Enter actual result	Enter actual result
Average	Calculate average	Calculate average

Average execution time = (Run 1 + Run 2 + Run 3) / 3.

Percentage improvement = ((Original average − Redesigned average) / Original average) × 100.

Use the same machine, Java version, workload and test procedure for both implementations. For more reliable benchmarking, warm up the JVM, alternate test order and use a Java benchmarking framework such as JMH where available.

F. Defend the Final Design (2 Marks)

Advantages of the redesigned system:

Reduces intermediate String creation.

Improves performance for repeated appending.

Uses mutable character storage.

Suitable for generating large numbers of tickets.

Produces the same final ticket content.

Limitations:

StringBuilder is not synchronized and should not be shared unsafely between threads.

Buffer resizing may require additional memory.

A final String object is still created when toString() is called.

For very small strings, the difference may be negligible and + may be more readable.

Complexity Comparison

Factor	Original String	Redesigned StringBuilder
Time	$O(n^2)$ character copying in the general repeated-growing-string case	O(n) amortized for appending n total characters
Intermediate objects	More temporary String objects	Fewer temporary objects
Space	Higher allocation pressure from intermediate strings	O(n) buffer space plus final String
Large input performance	Generally less efficient	Generally more efficient

Here n represents the total number of generated characters. The $O(n^2)$ comparison assumes each concatenation copies the previously accumulated string.

Final Result

The original railway ticket-generation system was analysed and redesigned using StringBuilder. The redesigned approach reduces repeated character copying and intermediate String allocations. It is a better engineering choice for generating large numbers of railway tickets, subject to validation with actual measurements.

MARK DISTRIBUTION

Question	**Assessed Area**	**Bloom's Level**	**Marks**
Q1	String, StringBuilder & StringBuffer	K4 – Analyse	6
Q2	ArrayList, LinkedList & ArrayDeque	K4 – Analyse	4
Q3	HashSet & the hashCode–equals contract	K4 – Analyse	6
Q4	HashMap, TreeMap & LinkedHashMap	K4 – Analyse	6
Q5	Generics, Comparable & Comparator	K4 – Analyse	7
Q6	Queue & PriorityQueue	K5 – Evaluate	8
Q7	Day-4 Design Challenge – Diagnose, Redesign & Validate	K5 – Evaluate	13
	TOTAL		**50**